Lecture 11

Security testing

Faculty of Information Technology
Hanoi University

Outline

- Quality vs. Security
- Security testing

References

- Gary McGraw, Software Security: Building Security In
- Dafydd Stuttard and Marcus Pinto, The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws
- Michael Howard and David LeBlanc, Writing Secure Code
- Laura Bell et al., Agile Application Security: Enabling Security in a Continuous Delivery Pipeline

Quality vs. Security

Quality vs. Security

Does high quality software equal secure software?

- High-quality software must inherently include security
- Security requires upfront specification, not post-development retrofitting
- Security in quality dimensions: It stands equally alongside performance, functionality, and maintainability
- Security testing is just as critical as testing for functional outputs

Common security vulnerabilities – Memory & Data

- **Vulnerability:** A weakness or flaw in software architecture, code, or configuration that attackers can exploit
- **Buffer Overflow:** Writing data that exceeds the allocated memory length (common in C/C++), causing crashes or unauthorized access
- **SQL Injection:** Injecting malicious SQL statements into input fields to manipulate or steal backend database records

Common security vulnerabilities – Web & Network

- **Cross-Site scripting (XSS):** Injecting malicious scripts into trusted websites to hijack sessions or execute code in other user's browsers
- **URL Manipulation:** Altering URL parameters or strings to bypass authorization and access hidden data
- **Spoofing:** Disguising communication (e.g., an IP address,…) to appear as a trusted, legitimate source

Security testing

Security testing

Traditional testing techniques:

- Standard Black/White box testing focuses only on expected inputs and outputs
- Security errors often don't violate basic functional requirements
- Traditional testing proves what software *can* do, not what it **shouldn't** do
- Effective security testing requires a specialized tools and an “attacker mindset”

Security testing

Penetration testing (Pen testing):

- **Concept:** Manual, exploratory testing simulating real-world malicious attacks
- **Objective:** Discover and exploit vulnerabilities to assess how easily a system can be accessed
- **Black box testing:** Zero internal knowledge; relies on external exploits (e.g., phishing, social engineering)
- **White box testing:** Full system/code access to uncover complex internal flaws (e.g., SQL injections)
- **Requirement:** Requires high expertise to assess system resilience against human-driven attacks

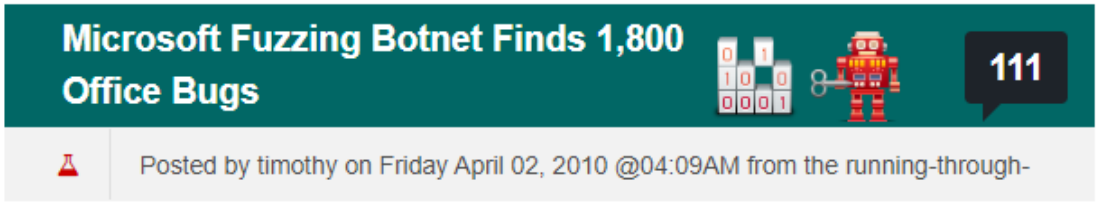
Security testing

Fuzzing (Fuzz testing):

- **Concept:** Automated brute-force injection of invalid or semi-valid data to trigger crashes
- **Objective:** Trigger unintended behavior or system crashes to uncover hidden security vulnerabilities
- **Black box random fuzzing:** Generates large volumes of completely random input data
- **Grammar-based fuzzing:** Mutates known valid data formats into unexpected edge cases
- **White box fuzzing:** Uses source code access and mathematical solvers to force unintended execution paths

Security testing

Fuzzing at Microsoft (2010)



Microsoft Fuzzing Botnet Finds 1,800 Office Bugs 111

Posted by timothy on Friday April 02, 2010 @04:09AM from the running-through-

CWmike writes

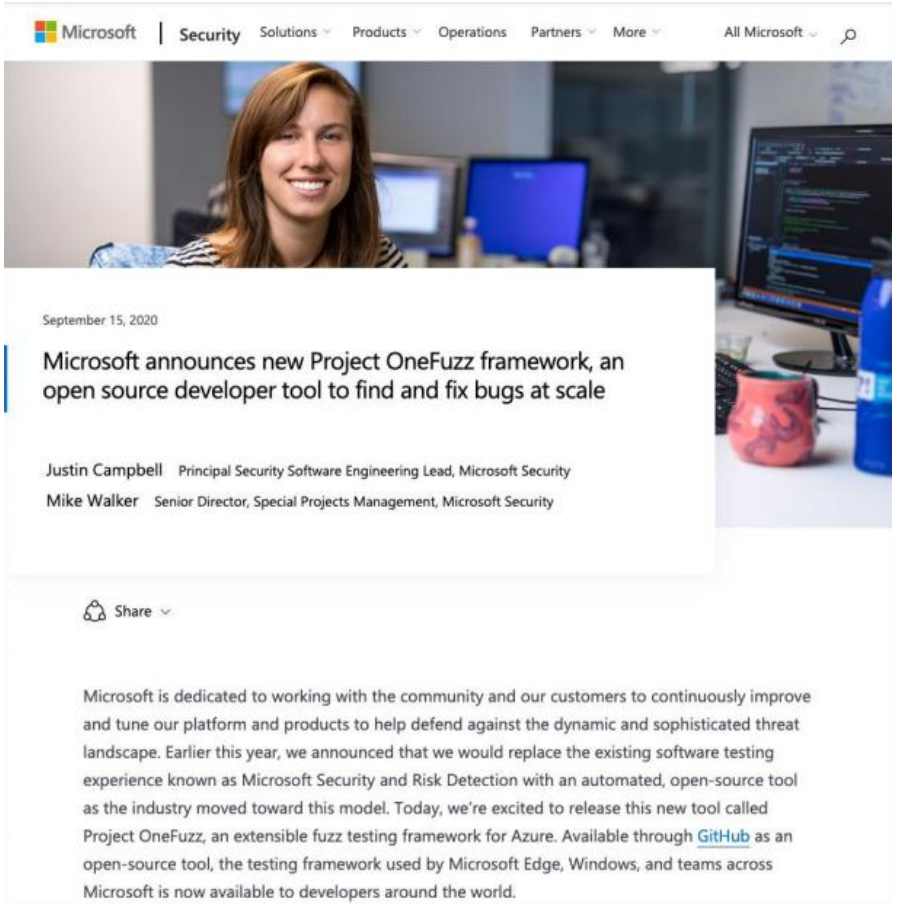
> "Microsoft uncovered more than 1,800 bugs in Office 2010 by tapping into the unused computing horsepower of idling PCs, a company security engineer said on Wednesday. Office developers found the bugs by running millions of 'fuzzing' tests, a practice employed by both software developers and security researchers, that searches for flaws by inserting data into file format parsers to see where programs fail by crashing. 'We found and fixed about 1,800 bugs in Office 2010's code,' said Tom Gallagher, senior security test lead with Microsoft's Trustworthy Computing group, who last week co-hosted a presentation on Microsoft's fuzzing efforts at the CanSecWest security conference. 'While a large number, it's important to note that that doesn't mean we found 1,800 security issues. We also want to fix things that are not security concerns.'"

Image Credit: https://developers.slashdot.org/story/10/04/02/0733221/Microsoft-Fuzzing-Botnet-Finds-1800-Office-Bugs

Security testing

Fuzzing Tools

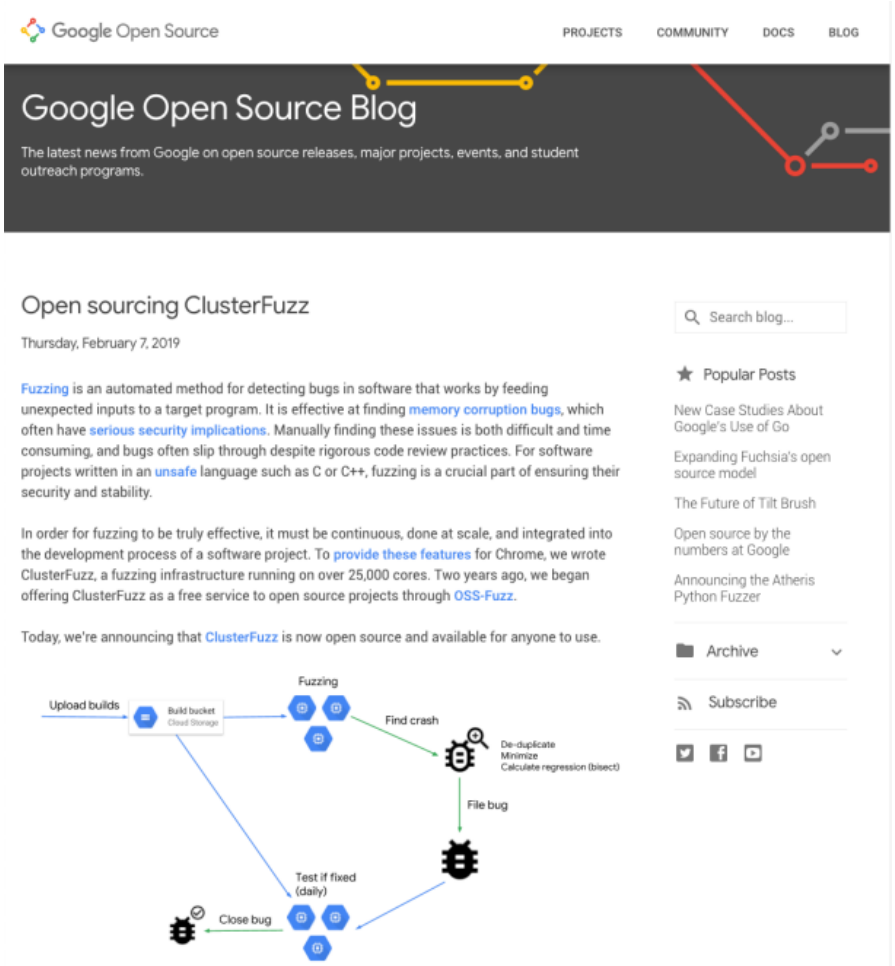
- Microsoft has open sourced **OneFuzz**
- OneFuzz is an Azure testing framework



Security testing

Fuzzing Tools

- Google has open sourced **ClusterFuzz**
- It has been used to find:
  - 27,000 chrome bugs
  - 28,000+ open-source project bugs



Application security in SDLC

- **The Reality:** No single security testing tool can find and fix *all* vulnerabilities
- **The Strategy:** Tools must complement each other across the Software Development Life Cycle (SDLC)
- **DevSecOps Goal:** Combine at least two distinct tools to inject security without slowing down development speed
- **Four methods:**
  - SAST (Static Application Security Testing)
  - DAST (Dynamic Application Security Testing)
  - IAST (Interactive Application Security Testing)
  - RASP (Runtime Application Self-Protection)

SAST

- **Concept:** “White-box” testing analyzing source code/binaries at rest (no execution)
- **SDLC Phase:** Early stages (IDE, code commit)
- **Key Capabilities:**
  - *Pros:* Finds hardcoded credentials, weak logic, SQL injections early (cheaper/faster fixes)
  - *Cons:* Cannot detect runtime errors; may produce false positives
- **Common Tools:** SonarQube, Checkmarx, Fortify, Veracode

DAST

- **Concept:** “Black-box” testing evaluating running apps from the outside-in, simulating real attacks
- **SDLC Phase:** Later stages (Staging, QA, Production)
- **Key Capabilities:**
  - *Pros:* Detects runtime issues (XSS, auth bypasses, server misconfigurations)
  - *Cons:* No access to source code; tests the application as a black box
- **Common Tools:** OWASP ZAP, Burp Suite, Invicti, Acunetix

IAST

- **Concept:** Hybrid approach operating *within* the app runtime to monitor interactions continuously
- **SDLC Phase:** Active during QA, automated UI, and functional testing
- **Key Capabilities:**
  - *Pros:* Finds flaws earlier than DAST. Pinpoints vulnerable code & verifies exploitability. Scans 3rd-party/open-source components
  - *Cons:* Not a complete replacement for DAST or penetration testing
- **Common Tools:** Contrast Security, Synopsys Seeker, Checkmarx IAST, HCL AppScan

RASP

- **Concept:** A *defense* tool integrating directly into the app server to monitor traffic & behavior
- **SDLC Phase:** Production (actively protects live applications)
- **Key Capabilities:**
  - *Pros:* Prevents runtime attacks (alerts, blocks requests, "virtually patches" apps). Deeper code-level visibility than a web application firewall
  - *Cons:* Not a "silver bullet"; it must complement, rather than replace, your overall testing strategy
- **Common Tools:** Imperva RASP, Contrast Protect, OpenRASP, Datadog

Advantages of security testing

- Identifies vulnerabilities before attackers exploit them
- Protects sensitive data (confidentiality, integrity, availability)
- Reduces financial and reputational risks
- Ensures compliance (GDPR, ISO 27001…)

Disadvantages of security testing

- Requires specialized skills and tools
- Time-consuming and expensive
- Cannot guarantee complete security
- May generate false positives due to tool’s mistake

General guidelines

General guidelines

- Think of all the possible things your code does
- Follow guidelines for your programming language and software type
  - Official tutorials and docs always recommend best practices
- Reuse trusted code (libraries, modules, etc.)
  - From reputable publishers
- Write good-quality, readable code
  - Logically organized and readable code also tend to be more secure

Best practices (Error Handling & Logging)

- Display generic error messages
  - Error messages should not reveal details about the internal state of the application
  - Do not include unnecessary information in error messages
- Handle all exceptions
  - Throwing exceptions can expose sensitive application data
- Log important user activities
  - Log data changes, user authentications
  - Do not log sensitive data such as passwords

Best practices (Error Handling & Logging)

Code exposure through error message

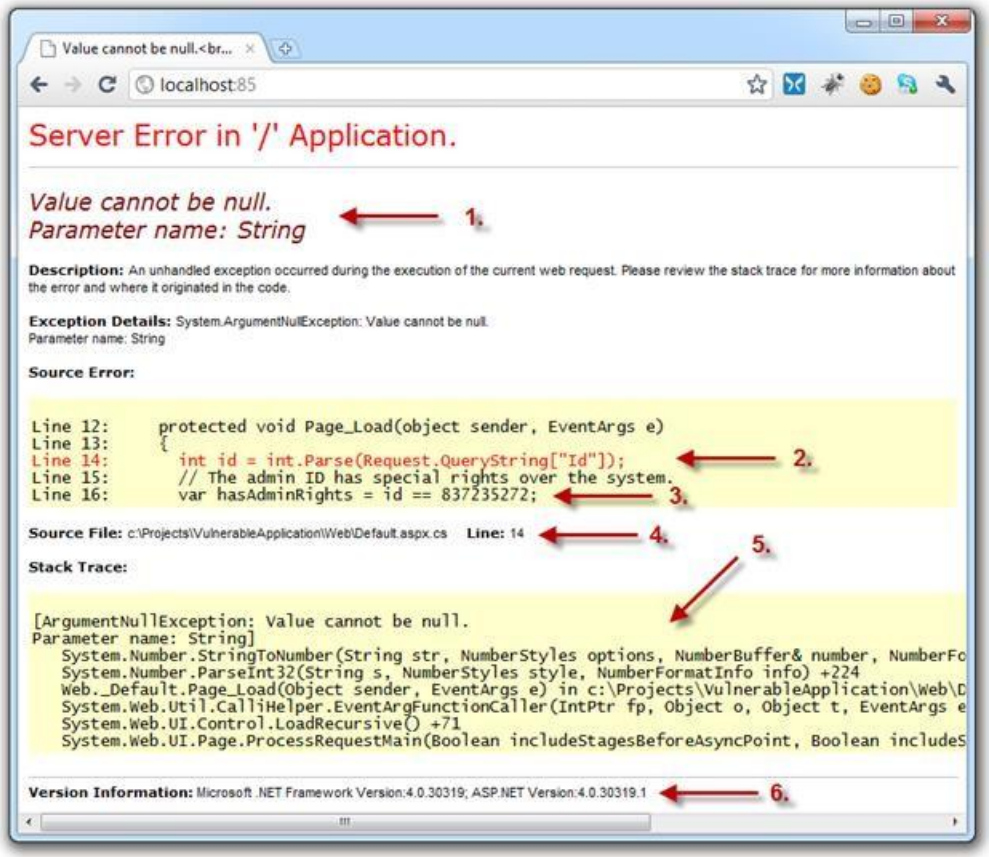


Image Credit: troyhunt.com

Best practices (Network & Permissions)

- Use HTTPS Everywhere
  - HTTPS protects all HTTP requests and responses from the Transport Layer
- Properly set file access/execution permissions
  - Windows “Security” tab in file/folder properties
  - chown, chmod commands in Linux

Best practices (Network & Permissions)

HTTP packet capture password

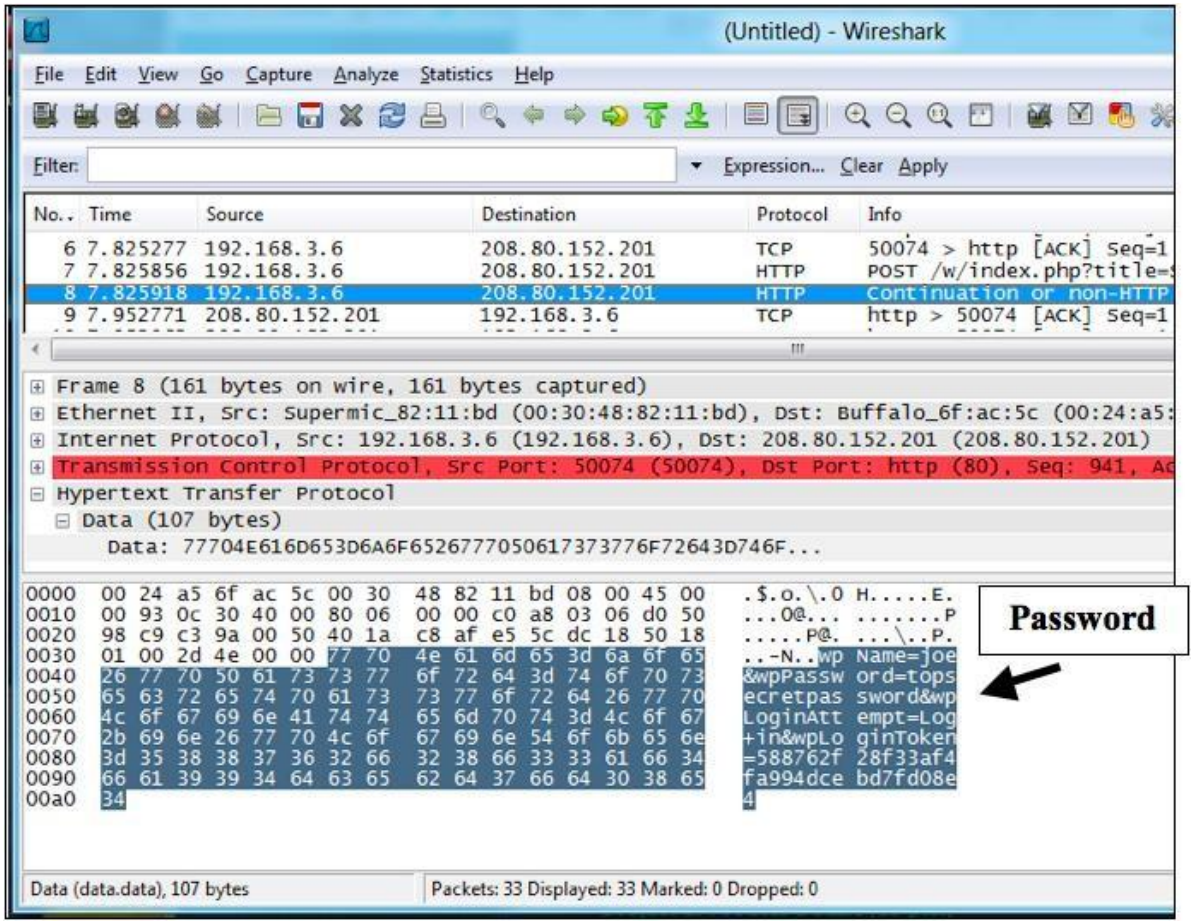


Image Credit: samsclass.info

Best practices (Credentials & Policies)

- Do not hard-code credentials
  - E.g. database passwords, admin passwords
  - Convenience leads to security flaws
- Enforce password policy
  - Do not allow users to use weak passwords
  - General rules for a strong password:
    - Long, mixing letters, digits & symbols
    - Not related to personal info
    - Avoid using dictionary words or meaningful phases

Best practices (Network & Permissions)

Hack tool: Memory Editor

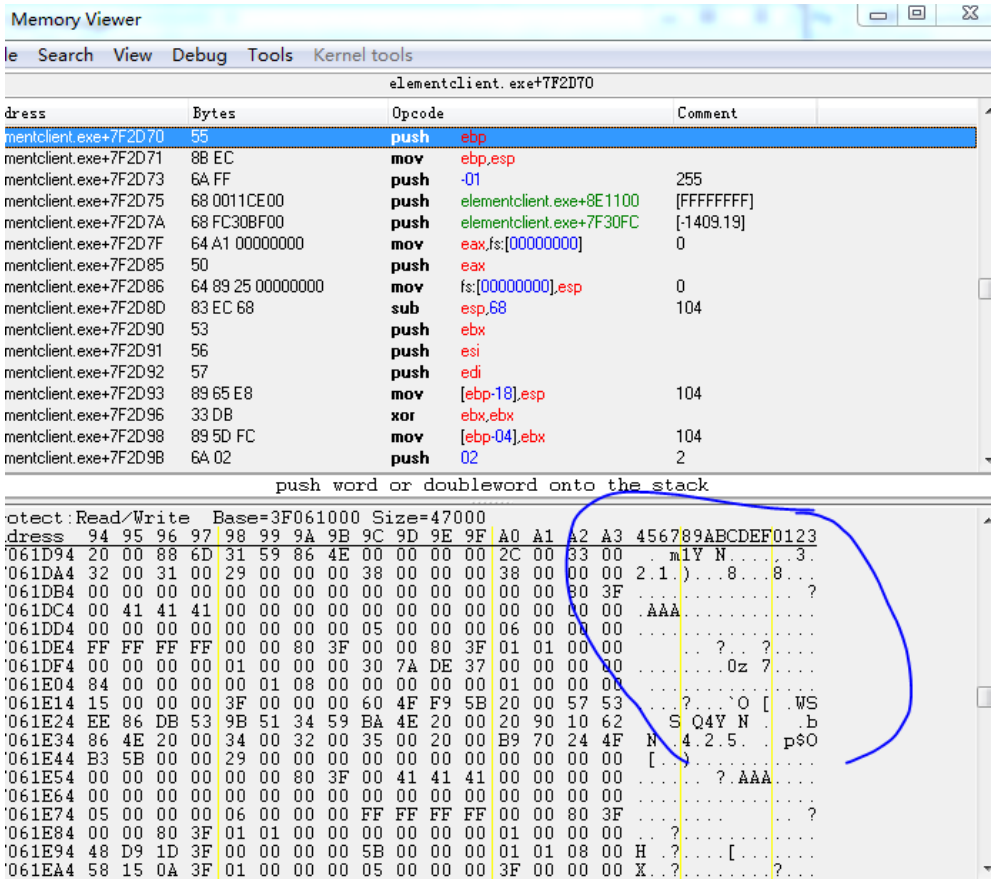


Image Credit: cheatengine.org

Best practices (Password Storage & Tokens)

- Store encrypted passwords
  - Strong, Salted hash
  - It’s not possible to restore a forgotten password
  - It’s only possible to reset a password
- Use tokenization to reduce data exposure
  - A token is an encrypted string which is linked to user session on the server
  - A token is a proof of authentication

Best practices (Password Storage & Tokens)

Hashed passwords

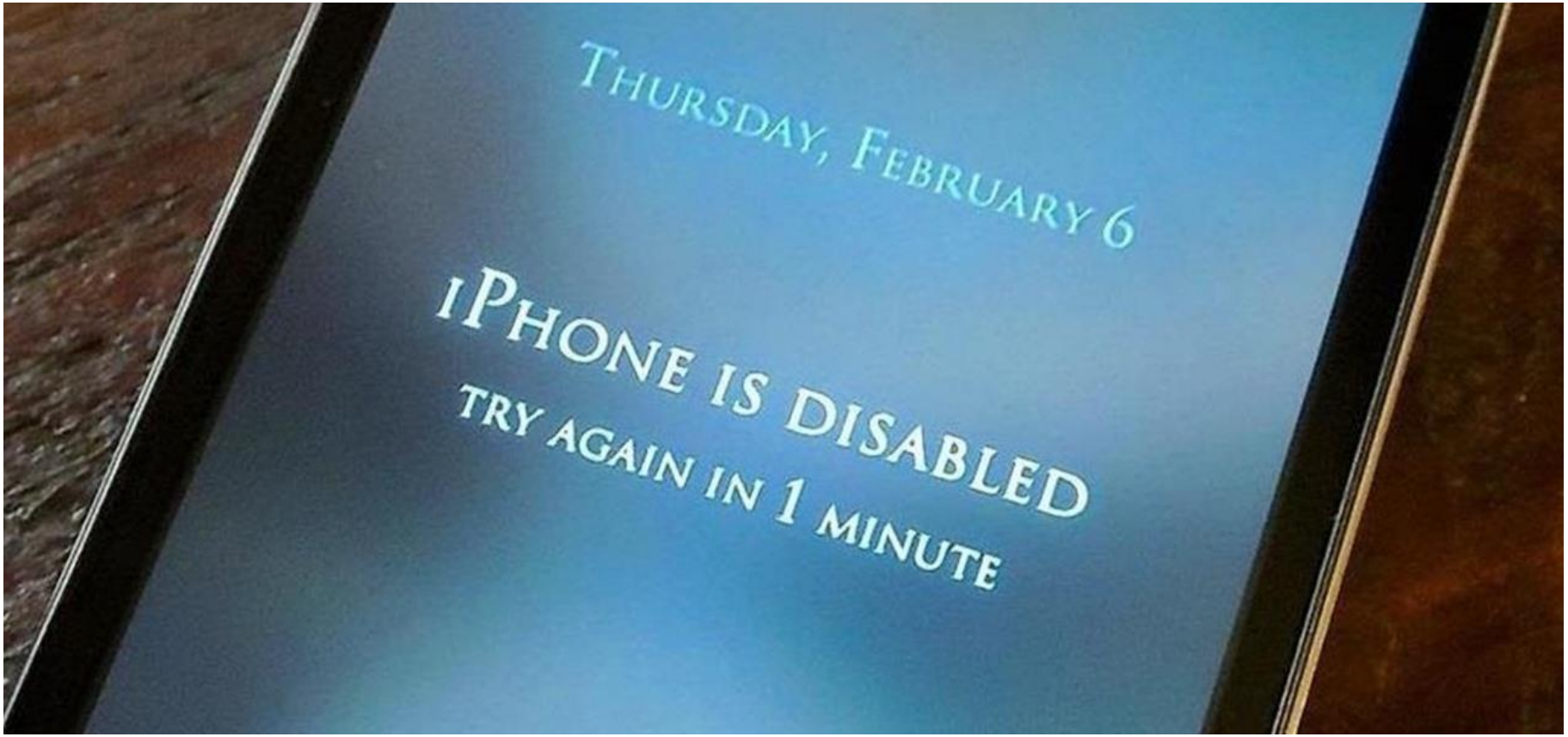
Input	Hashed	Algorithm	Salted
namtt	$2a$12$N4K2CVkRH7ljycgYa9plN.8OJG/D7nKXAGS.6AvEd5OaasmT0dWWS	Bcrypt	Yes
namtt	d73af99438d4de8a492f0236e72a3909	MD5	No
namtt	474050e8f21a7ee1b7fc512b5301b0bd385fb042	SHA-1	No
namtt	390c50c33ce1a40c749d0753858eabed3a3296ba3b15233c17c5a9550f35b7ef	SHA-256	No
namtt	$argon2i$v=19$m=16,t=2,p=1$a3l5aEtHQmoxd1JBcmYwNw$AD8Ona8E4JUy1l0fHB1SAg	Argon2i	Yes

Best practices (Access Control)

- Account Lockout
  - Lock account after several incorrect passwords
  - Helps protect against Brute Force Attacks
- Run applications with minimal privileges
  - Optimization is about giving only what is necessary
- Set cookies/session/password expiration
  - Any kind of encryption is only valid for a certain amount of time (the time it takes to break it)

Best practices (Access Control)

Security feature: account lockout



Disable a feature after some failed attempts

Image Credit: ios.gadgethacks.com

Best practices (Input Validation & Trust)

- Validate all inputs
  - Uploaded files, Input fields, The source of inputs (e.g. Referrer URL, the source application, the input device…)
- Choose safe defaults
  - Default values such as default passwords, default privileges should be safe
- Prefer whitelist over blacklist
  - It is safer to block all unexpected things
- In web application, do not trust client-side validation
  - Client-side validations can be bypassed easily

Best practices (Preventing Injection, Bots & DDoS)

- Use up-to-date database access methods
  - E.g. Prepared Statements to prevent SQL Injection
- Use appropriate data encoding
  - Apply context-specific encoding to prevent XSS Attacks
- Generate and use trusted token (Captcha)
  - Prevent automated submissions
  - Secret is the foundation of security
- Keep bottlenecks as light as possible
  - Optimize handlers & login servers to minimize DDoS impact

Summary

- Quality vs. Security
- Security testing
- General guidelines